CommPulse

Project Team

Haris Sohail 21I-0531
Zain Rizwan 21I-2500
Sufian Sajjad 21I-0688

Session 2024-2025

Supervised by

Mr. Aqib Rehman

Co-Supervised by

Dr. Arshad Islam

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2024

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The purpose of this chapter is to outline the objectives, background and technical details of the project CommPulse. This document is a guide for the stakeholders of this project.

Quality communication serves as one of the cornerstones in many fields such as project management, education, and other corporate environments[3]. Virtual meetings have gained much popularity after the COVID-19 period. One of the most important reasons of this popularity is the increased efficiency of resources while using the virtual mode in communication[1]. While there are resources to improve communication quality, there is a significant lack in quality monitoring tools of virtual meetings. This gap motivates us to make online meetings more interactive, fun and engaging.[2]

CommPulse aims to solve these problems by monitoring the meeting quality in real-time as well as after the meeting has ended.

1.1 Problem Statement

In today's corporate and educational environments effective communication is becoming increasingly important to achieve better results. However, many lack the essential traits of quality exchanges. If we follow the key principles of productive communication, there would be significant reduction in cost, time and other resources. Although there are many resources that help to improve communication standards, but there is a noticeable lack in the amount of quality monitoring tools for online meetings and conferences. This lack encourages our team to develop a streamlined system that monitors the quality of online meetings. The system will track important communication traits and give real-time feedback to the user. This feedback will later be presented to the user in an easy-to-understand manner. This solution will enhance productivity, relationships, employee morale, decision-making and many other benefits that come with good and effective communication. In conclusion, the system will be significant in removing bad

traits of dialogue in many sectors.

1.2 Scope

The project is aimed to develop a software solution for monitoring the quality of online meetings on the meeting platform Microsoft Teams. The primary responsibility of the tool is to continuously monitor incoming audio stream from the meeting. It also aims to provide monitoring services after the meeting has ended. The quality metrics that the product will cover are sentiments, active participation, clarity, adherence to the agenda, and meeting minutes. During the meeting, these metrics will be monitored on the conversation-level. After the meeting these metrics will be monitored on the speaker-level. The system will be developed to support a wide range of scenarios including corporate meetings, educational lectures, as well as informal meetings. Furthermore, a user-friendly interface will be developed for real-time as well as post meeting analysis. Easy-to-understand graphs and figures will be shown to the user in real-time so that they can review the analysis and handle meetings efficiently. The boundaries of the system consist of real-time and posttime audio processing for different quality metrics, excluding video and other forms of real-time communication.

1.3 Modules

Following are the modules we plan on doing in our FYP:

1.3.1 UI/UX design

This module will focus on creating a UI design of the application. This will align the project team, and will play an important role in defining the outline of the system as whole.

1. User Management screens
2. Home screen
3. Graphs/Figures showing meeting quality metrics (real-time)
4. Post meeting screen
5. Graphs/Figures showing meeting quality metrics (post-time)
6. Meeting history screen

1.3.2 Front-end development

In this module, the UI design will be converted into a working front-end desktop application. The front-end will be made using ElectronJS in combination with ReactJS. This will be independent of the back-end.

1. User management and authentication
2. Home screen
3. Start meeting screen
4. Audio capturing using API
5. Ongoing meeting management
6. Post meeting page
7. Post-time plots showing meeting quality metrics including time-stamped highlights using data from live transcript
8. Meeting history using data from live transcript

1.3.3 Meeting Quality Monitoring System Back-End

This will be the core module of the system, and it will combine multiple audio processing AI models to monitor the metrics in real-time as well as post-meeting. A basic UI will be developed for the back-end using Streamlit/Dash/Gradio so that the development is modular, and testing is easy

1. Using API to distinguish speakers
2. Quality metric 1: Sentiments
3. Quality metric 2: Active Participation
4. Quality metric 3: Clarity
5. Quality metric 4: Adherence to the agenda
6. Quality metric 5: Time-stamped highlights

1.3.4 Meeting History Database

In this module, a relational database will be developed to store detailed meeting analytics. The database will hold information at the session level as well as speaker level. This includes the quality metrics discussed above.

1. Session Level Data Storage
2. Speaker Level Data Storage

1.3.5 Integration of front-end and back-end

This module will combine the front-end modules and the back-end modules to work flawlessly. Web sockets will be used to send the audio data from the front-end to the back-end in real-time.

1. Audio capturing using API
2. Real-time plots using back-end model's predictions
3. Post-time plots of meeting quality metrics including time-stamped highlights using model's predictions
4. Meeting history management

1.4 User Classes and Characteristics

This section defines the user classes with their characteristics. These anticipate the users who will use this product. Their pertinent characteristics are defined as follows:

Table 1.1: User Classes and Characteristics

User class	Description
Admin	The Admin can track live meetings and perform post-meeting analysis. They can view metrics live and manage user roles. Usually, a project manager or a technical staff member plays the administrative role, aiming to ensure that meetings function well. They own complete control over both live meeting monitoring and the analyses of meetings after they happen. The admin interface will give users real-time feedback, along with the ability to operate user management and set configurations. Approximately 10 percent of users will have admin privileges.
Regular User	Only after meetings have ended can Regular Users analyze what transpired in the meetings and evaluate metrics. Monitoring live meetings is not possible for them. Employees, students, and participants are the usual users who have to reassess communication quality once the meeting is over. They will use the platform mainly to assess post-meeting reports and summaries. Users anticipated to engage with the system after meetings should do so, 1-2 times per week. Only post-meeting data along with insights are accessible to this user class; there are no live monitoring features. Around 90 percent of the participants will be in this specific category.
Guest User	An external participant at the meeting with limited access to the system features is described as a guest user. The guests may join meetings, but they might not have access to post-meeting analytics or live monitoring. They only take part occasionally, solely in meetings they've been asked to join and use temporary access for. According to the system design, guests may or may not need a login or an ongoing profile. They usually take part through a link or invitation for designated meetings. Approximately 5-10 percent of users might be guests.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-cases

The use-case diagram shown below gives a clear understanding of the use-cases of the system CommPulse.

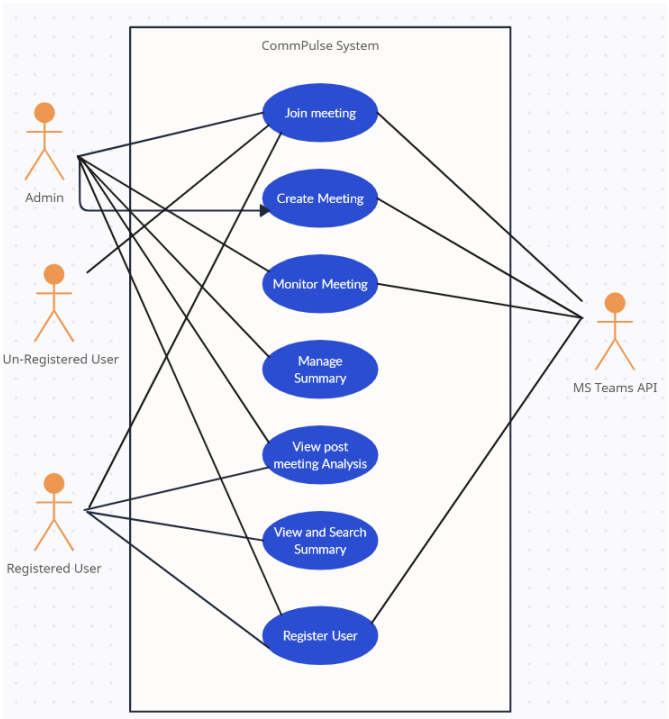

Figure 2.1: Use-case Diagram

2.2 Event Response Table

This section presents the event response table. CommPulse is a real-time system with most of it's functionality at the back-end. The system responses to user events in the following ways:

Table 2.1: Event Response Table

Event Number	**Event**	**System State**	**System Response**
1	User clicks "Start Meeting"	Pre-Meeting Setup	Prompt user to enter meeting title, agenda, and topics.
2	Meeting starts	In-Meeting	Start tracking metrics (e.g., sentiment, participation, agenda adherence) in real-time. Show a live graphical analysis on the front-end.
3	Meeting ends	Post-Meeting Analysis	Compile meeting summary and time-specific topics discussed. Show these to the user in the post-meeting screen.
4	User selects a specific time in the meeting in the post-meeting screen	Post-Meeting Analysis	Display specific highlights and topics from the selected timestamp.
5	User edits meeting summary	Post-Meeting Analysis	Let the user edit and update the meeting summary and overwrite the old summary.
6	User views graphical analysis of a specific meeting	Post-Meeting Analysis	Prompt the user to enter the meeting id and show a speaker-specific graphical analysis on the metrics: Sentiments Analysis, Agenda Adherence, Clarity, Active Participation.

7	User views meetings	Post-Meeting Analysis	Prompt the user to search a meeting using meeting title or ID. User can also search a meeting using the quality metrics such as Sentiments, Agenda Adherence, Active Participation, Clarity. The user can also use dates and time length of the meetings to search.
8	User opens a meeting	Post-Meeting Analysis	Upon opening a meeting user will be presented with the editable meeting summary, as well as the time-specific topics segregation of the meeting.

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature. It is just one possible way to arrange them. Other organizational options include arranging functional requirements by use case, process flow, mode of operation, user class, stimulus, and response depend on what kind of technique has been used to understand functional requirements. Hierarchical combinations of these elements are also possible, such as use cases within user classes.

2.3.1 UI/UX design

Following are the requirements for module 1:

1. **FR1**: The system shall provide user management screens for registration, login, and profile management.
2. **FR2**: The system shall present a home screen with options for starting or joining meetings, accessing meeting history, and user settings.
3. **FR3**: The system shall display real-time graphs and figures representing meeting quality metrics, including sentiment analysis and engagement levels.

4. **FR4**: The system shall provide a post-meeting screen that provides summarized and detailed meeting statistics.
5. **FR5**: The system shall generate post-time graphs showing key metrics such as clarity, adherence to the agenda, and speaker engagement.
6. **FR6**: The system shall display a meeting history screen where users can view data from previous meetings and detailed analytics

2.3.2 Front-end development

Following are the requirements for module 2:

1. **FR1**: The system shall provide user management and authentication functionality for user login and profile management.
2. **FR2**: The system shall display a home screen with options to manage meetings, user profiles, and settings.
3. **FR3**: The system shall include a start meeting screen, capturing necessary meeting details.
4. **FR4**: The system shall capture audio data from the user's device through an API for real-time analysis.
5. **FR5**: The system shall allow the user to manage ongoing meetings, including displaying real-time participant data and meeting metrics.
6. **FR6**: The system shall display a post-meeting page showing the meeting results, statistics, summary and post-meeting analysis.
7. **FR7**: The system shall generate and display post-time graphs showing meeting quality metrics and time-stamped highlights using live transcripts.
8. **FR8**: The system shall maintain a meeting history page showing detailed session data and time-stamped highlights from past meetings

2.3.3 Meeting Quality Monitoring System Back-End

Following are the requirements for module 3:

1. **FR1**: The system shall use an API to identify and distinguish speakers during the meeting.

2. **FR2**: The system shall analyze and display real-time sentiment metrics for the whole meeting.

3. **FR3**: The system shall monitor active participation for all participants and display engagement scores based on some given criteria.

4. **FR4**: The system shall monitor adherence to the meeting agenda and provide feedback after the session about how much effective the meeting was.

5. **FR5**: The system shall generate time-stamped highlights during the meeting to capture key moments for post-meeting analysis in a form of summary.

2.3.4 Meeting History Database

Following are the requirements for module 4:

1. **FR1**: The system shall store session-level data. e.g (start time, end time, participants, and agenda).

2. **FR2**: The system shall store speaker-level data, including metrics such as sentiment analysis, engagement, clarity, and time-stamped highlights for post-meeting analysis.

2.3.5 Integration of Front-End and Back-End

Following are the requirements for module 5:

1. **FR1**: The system shall capture audio from the front-end and transmit it to the back-end in real-time for analysis using an API.

2. **FR2**: The system shall display real-time plots of meeting quality metrics generated by back-end models.

3. **FR3**: The system shall generate and display post-time plots showing meeting metrics and time-stamped highlights.

4. **FR4**: The system shall retrieve and display meeting history data using the back-end database, including detailed post-meeting reports and analysis

2.4 Non-Functional Requirements

This section specifies nonfunctional requirements. These quality requirements highlight the specific criteria that can be used to judge the operation of a system, rather than specific behaviors.

2.4.1 Reliability

REL-1: The MTBF (Mean Time Between Failures) of the system shall be set to 1,000 operational hours indicating any unresolved system failure impacting major services.

REL-2: The system shall minimize the storage loss of meeting data to less than 1 minute during system failure.

REL-3: Within one minute of a system failure, the error logs shall be created, and recovery will take place automatically in less than five minutes.

REL-4: If the system fails it shall recover automatically with the user not needing to pause ongoing meetings. If a meeting is cut short because of an issue it shall pick up where it left off.

2.4.2 Usability

USE-1: Users shall reach post-meeting reports via the dashboard in just 5 clicks.

USE-2: A user shall initiate live meeting tracking as soon as the meeting has started.

USE-3: Help documentation and tooltips shall be present on every screen to help users execute key tasks (such as viewing results and post-meeting summary).

USE-4: Within 30 minutes of engagement with the interface users shall execute basic analysis tasks.

2.4.3 Performance

PER-1: For up to two-hour meetings, the analysis shall be completed in five minutes.

PER-2: Up to 100 users shall be handled simultaneously by this system and performance remains stable.

PER-3: The system shall ensure instant feedback in 10 seconds during live meeting tracking over a 20 Mbps or faster Internet connection.

PER-4: The system shall perform real-time monitoring of audio data with a most exces-

sive delay of 15 seconds while accommodating meetings containing up to ten attendees over a 20 Mbps or faster Internet connection.

2.4.4 Security

SEC-1: The system shall require multiple authentication methods for all admin users using a verification process with a password.

SEC-2: Live meeting monitoring along with user management and post-meeting reports shall be available to admins. Regular Users shall have access to review meeting analysis only Role-Based Access Control (RBAC).

SEC-3: The system shall support a connection protocol of TLS 1.2 or greater between the client and the server.

2.4.5 Maintainability

MAINT-1: To keep the system running efficiently changes in one module (such as sentiment analysis) shall affect fewer than 10% of the other modules.

MAINT-2: The documentation for codebase updates shall take place on schedule with the new features noted in 2 days.

MAINT-3: No system downtime for users shall be allowed during any meetings with updates and bug fixes deployed.

2.5 Domain Model

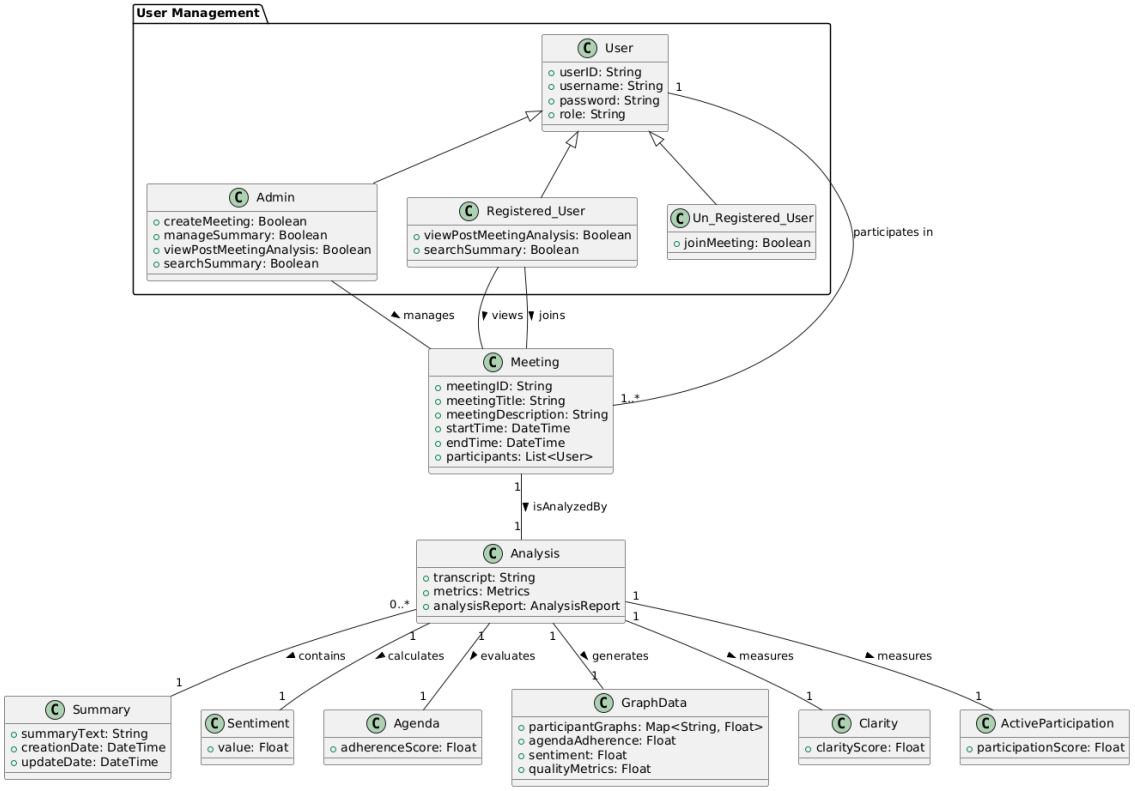

Figure 2.2: Domain Model Diagram

Chapter 3

System Overview

This section provides general description of the functionality, context, and design of CommPulse. CommPulse is a real-time meeting monitoring system which is destined to be a desktop application.

3.1 Architectural Design

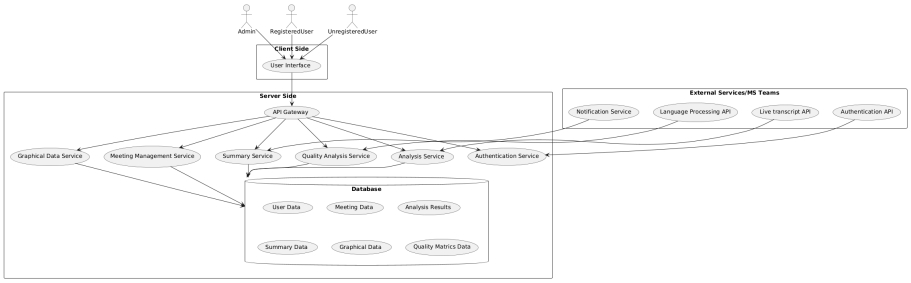

Figure 3.1: Client Server Architecture

3.2 Design Models

3.2.1 Design Models for Object-Oriented Development Approach

3.2.2 Activity Diagram

Activity diagrams describe the flow of control or the sequence of actions in a system. Below are the activity diagrams used in the system:

3.2.2.1 Create Meeting:

This activity diagram illustrates the process of creating a meeting in CommPulse.

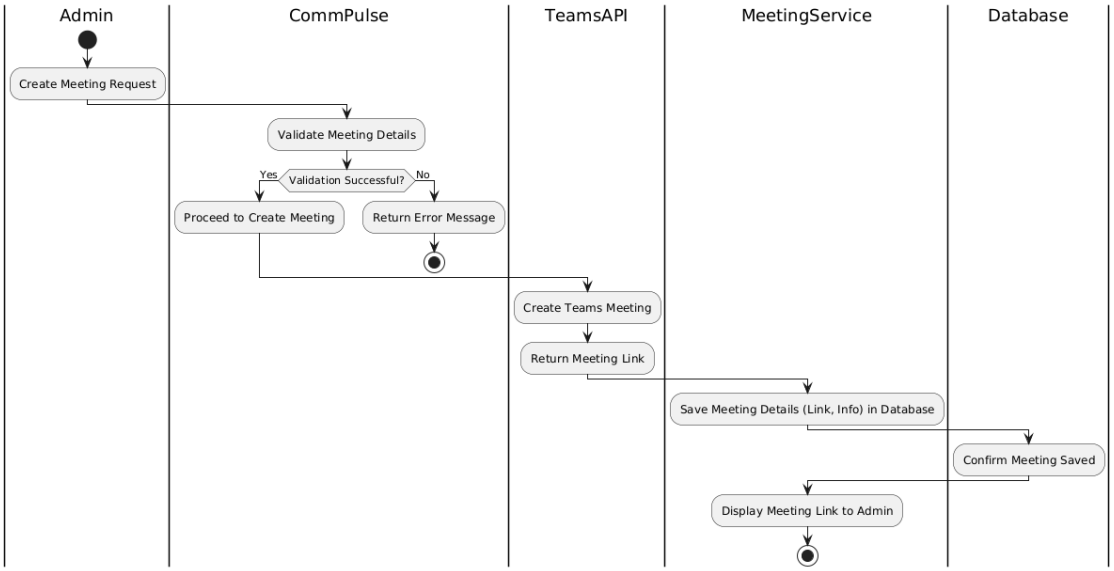

Figure 3.2: Creating a meeting

3.2.2.2 Join Meeting:

This activity diagram illustrates the process of joining a meeting in CommPulse.

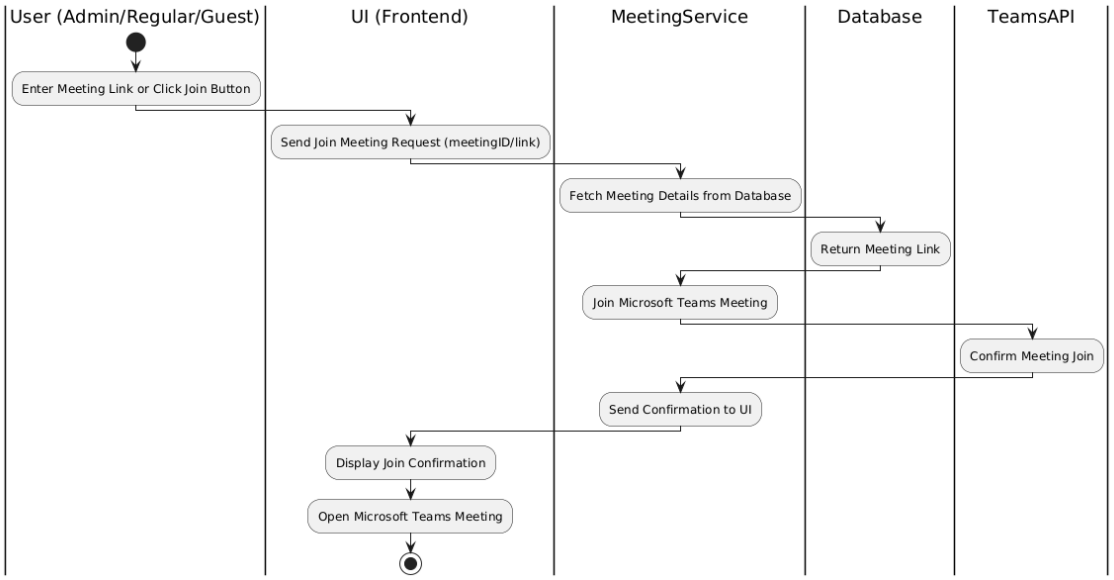

Figure 3.3: Joining a meeting

3.2.2.3 Manage Summary:

This activity diagram illustrates how a user manages the meeting summary in CommPulse.

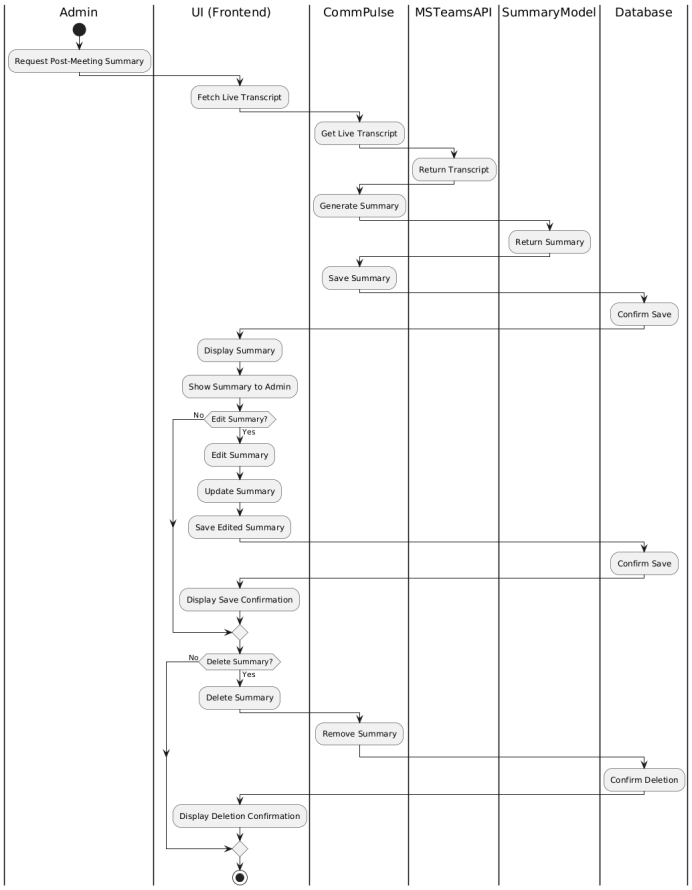

Figure 3.4: Managing the meeting summary

3.2.2.4 Monitor Meeting:

This activity diagram illustrates the process of monitoring a meeting in CommPulse to gather insights and data.

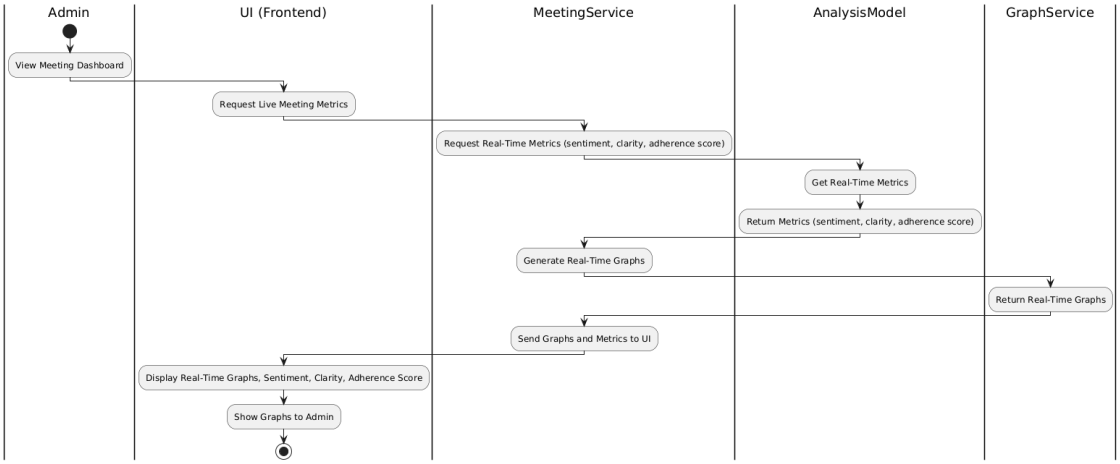

Figure 3.5: Monitoring a meeting

3.2.2.5 View Post Meeting:

This activity diagram illustrates how Admin can view the meeting summary and insights after the meeting concludes in CommPulse.

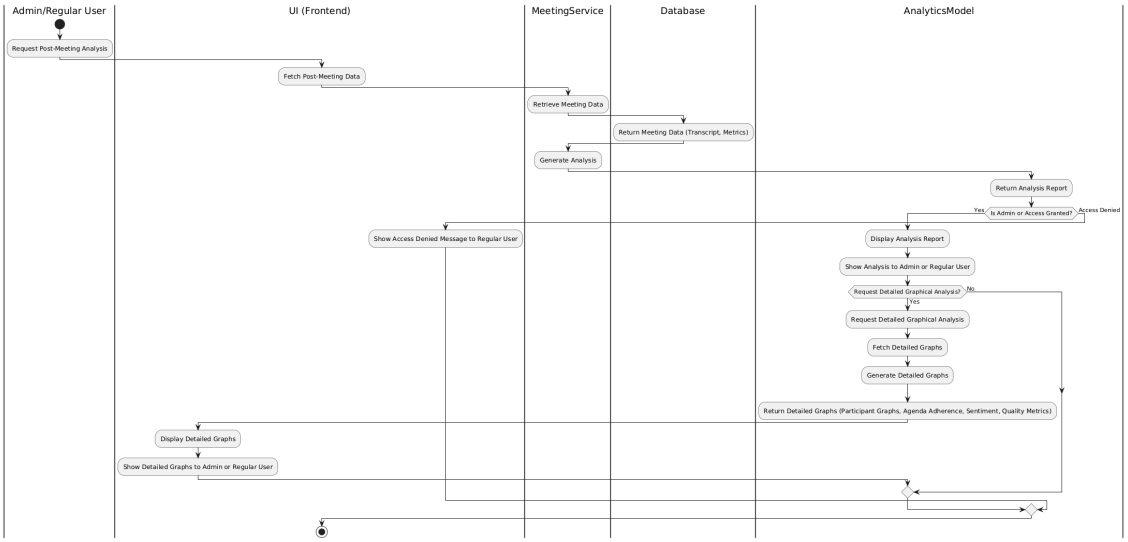

Figure 3.6: Viewing post-meeting summary

3.2.2.6 View and Search Summary:

This activity diagram illustrates the process of viewing and searching through meeting summaries in CommPulse.

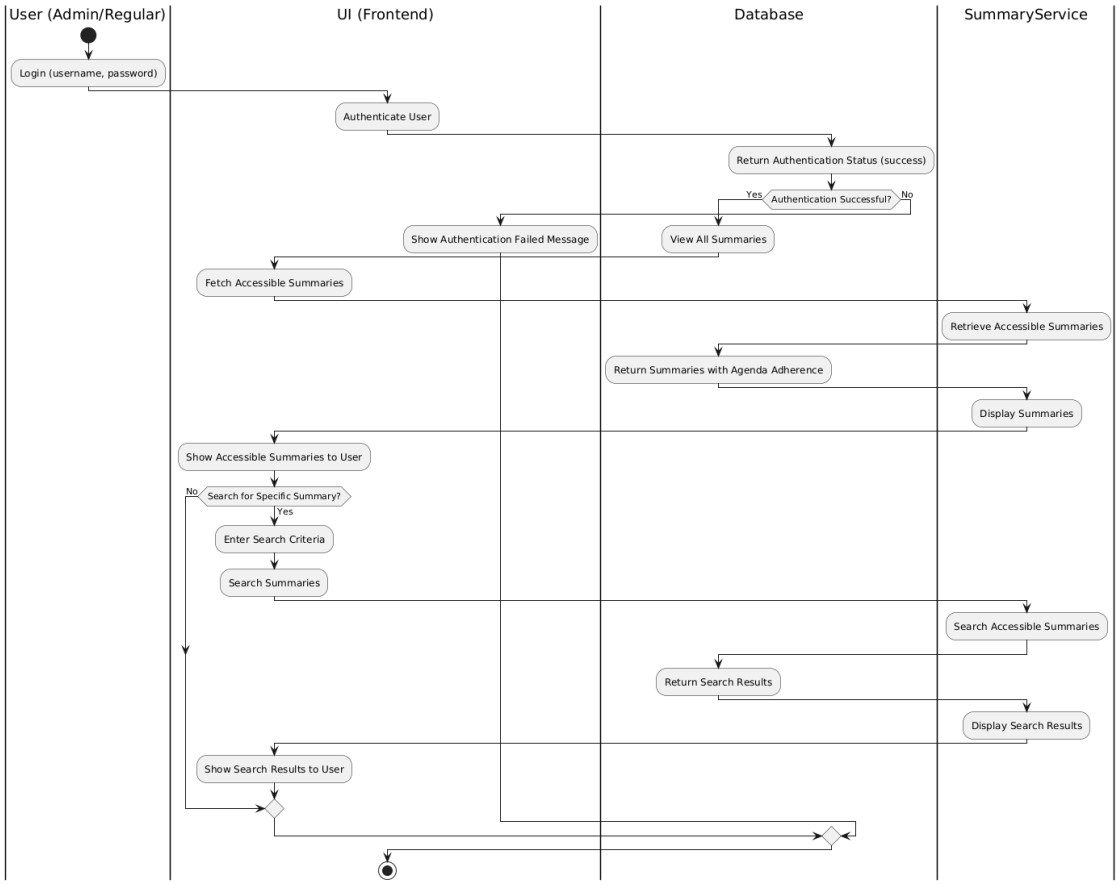

Figure 3.7: Viewing and searching meeting summaries

3.2.3 Class Diagram

A class diagram showing the relationships between system classes in CommPulse.

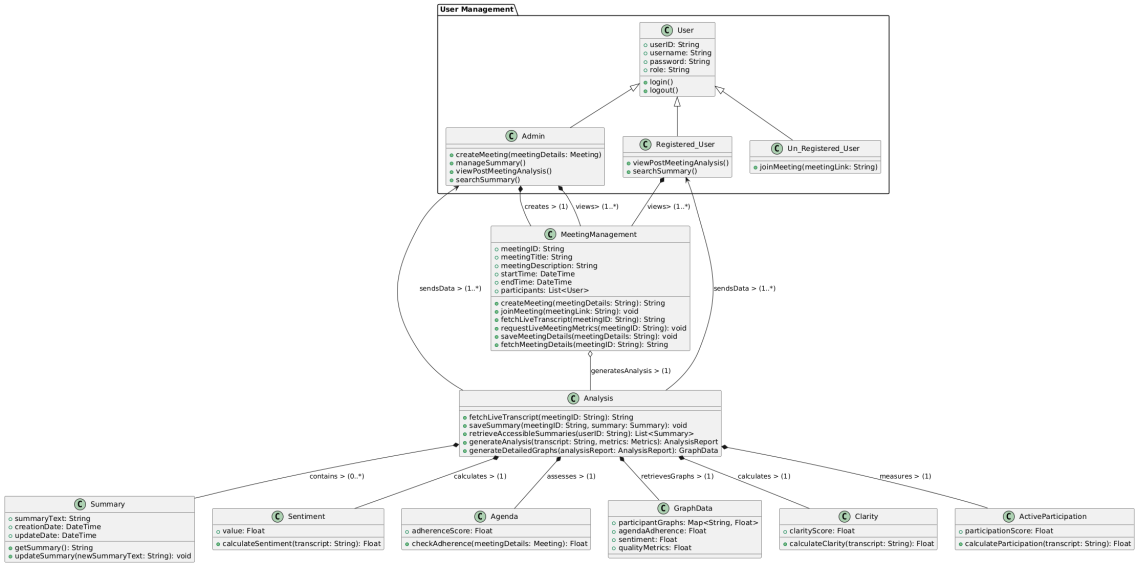

Figure 3.8: Viewing and searching meeting summaries

3.2.4 Class-Level Sequence Diagram

A class-level sequence diagram depicting the interaction between classes over time. Sequence diagrams in CommPulse depict the interactions between objects over time to accomplish specific functionalities. They show how objects communicate in a particular sequence of events. Below are the sequence diagrams for key processes in the system.

3.2.4.1 Create Meeting Sequence:

This sequence diagram illustrates the interaction between objects during the process of creating a meeting in CommPulse.

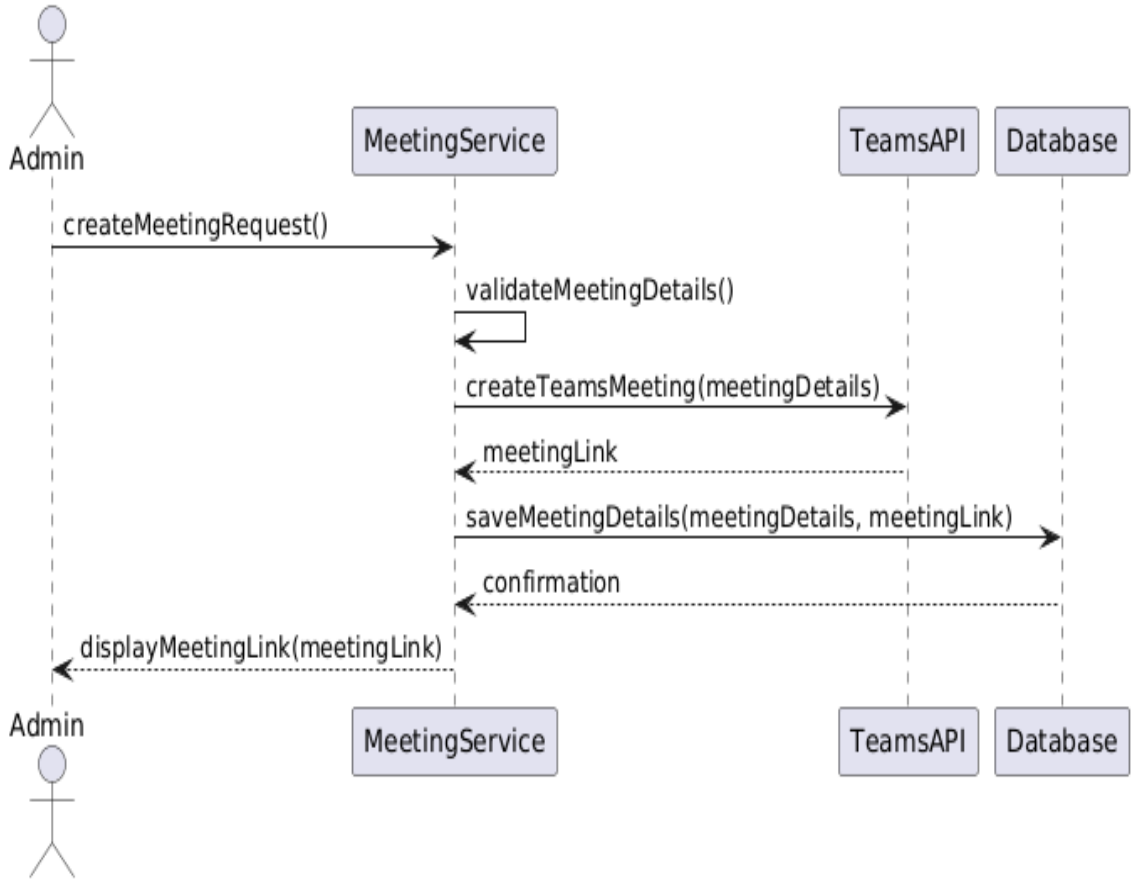

Figure 3.9: Creating a meeting - Sequence Diagram

3.2.4.2 Join Meeting Sequence:

This sequence diagram illustrates the interaction between objects during the process of joining a meeting in CommPulse.

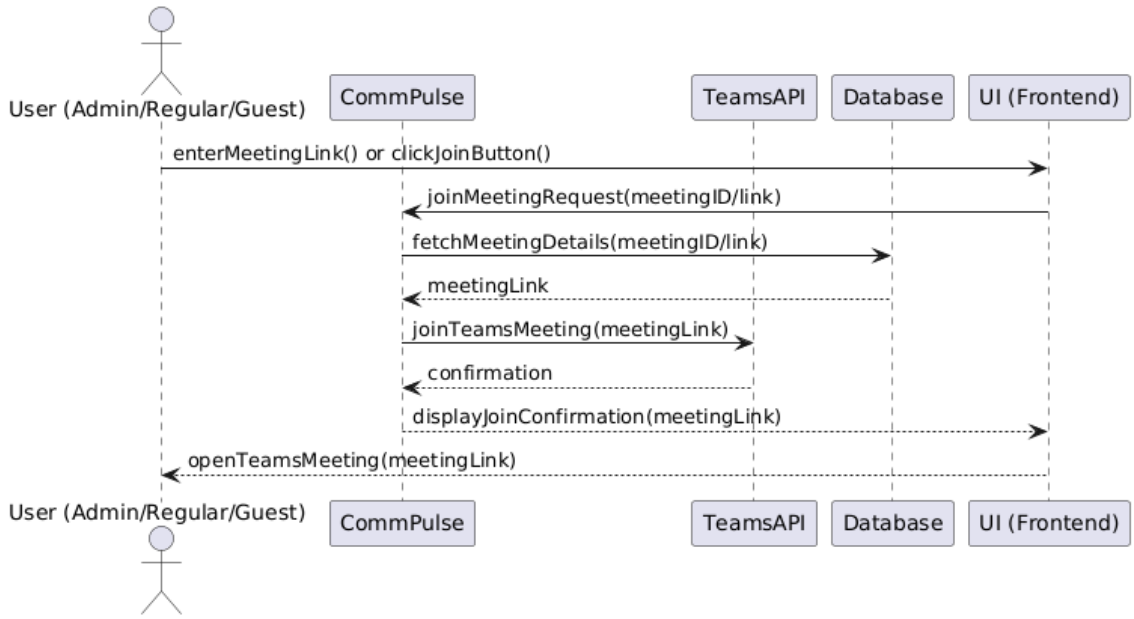

Figure 3.10: Joining a meeting - Sequence Diagram

3.2.4.3 Manage Summary Sequence:

This sequence diagram illustrates the interaction between objects when managing a meeting summary in CommPulse.

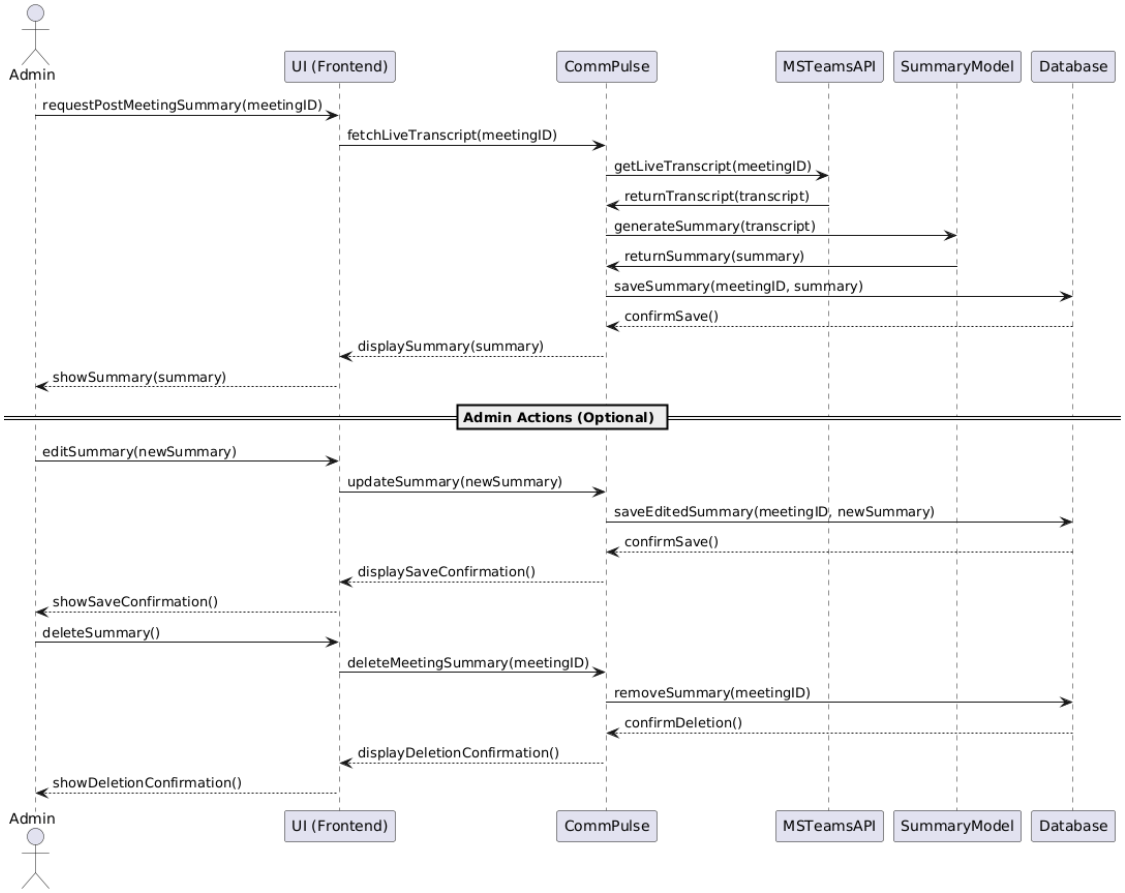

Figure 3.11: Managing a meeting summary - Sequence Diagram

3.2.4.4 Monitor Meeting Sequence:

This sequence diagram shows the interaction between objects during the monitoring of a meeting in CommPulse, collecting insights and data.

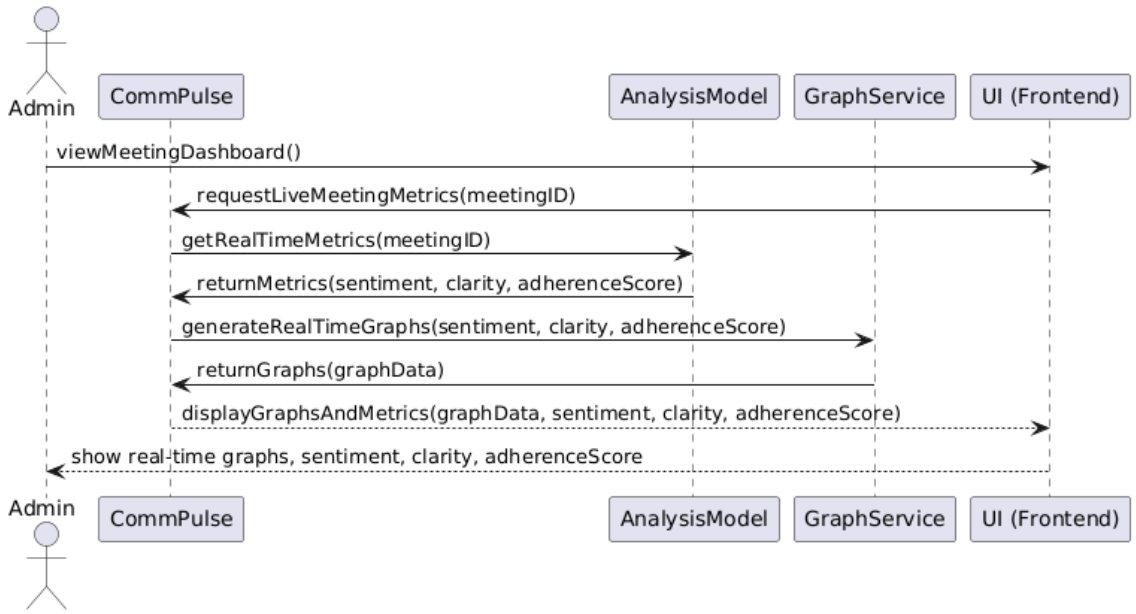

Figure 3.12: Monitoring a meeting - Sequence Diagram

3.2.4.5 View Post Meeting Sequence:

This sequence diagram illustrates how an Admin can view the meeting summary and insights after a meeting concludes in CommPulse.

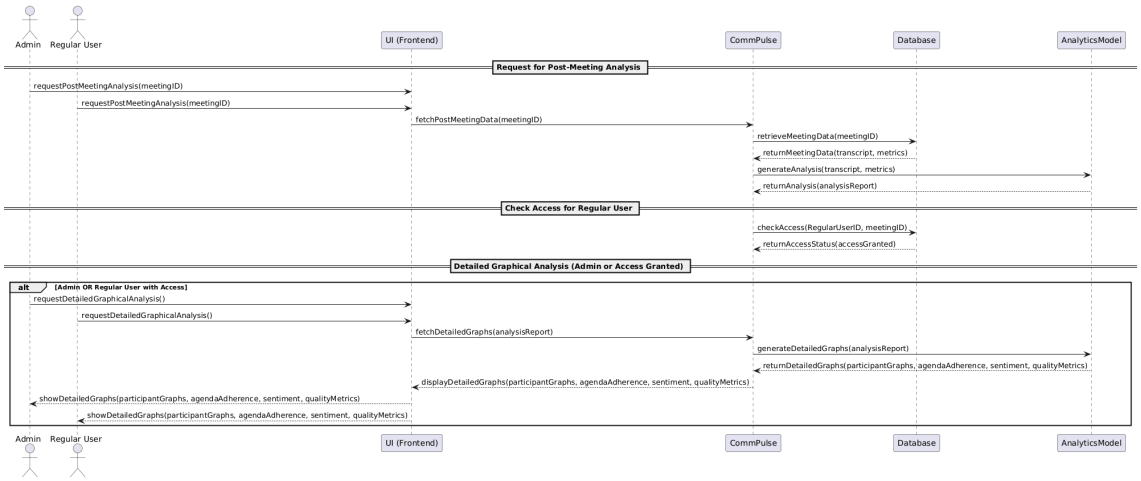

Figure 3.13: Viewing post-meeting summary - Sequence Diagram

3.2.4.6 View and Search Summary Sequence:

This sequence diagram illustrates the interactions involved in viewing and searching through meeting summaries in CommPulse.

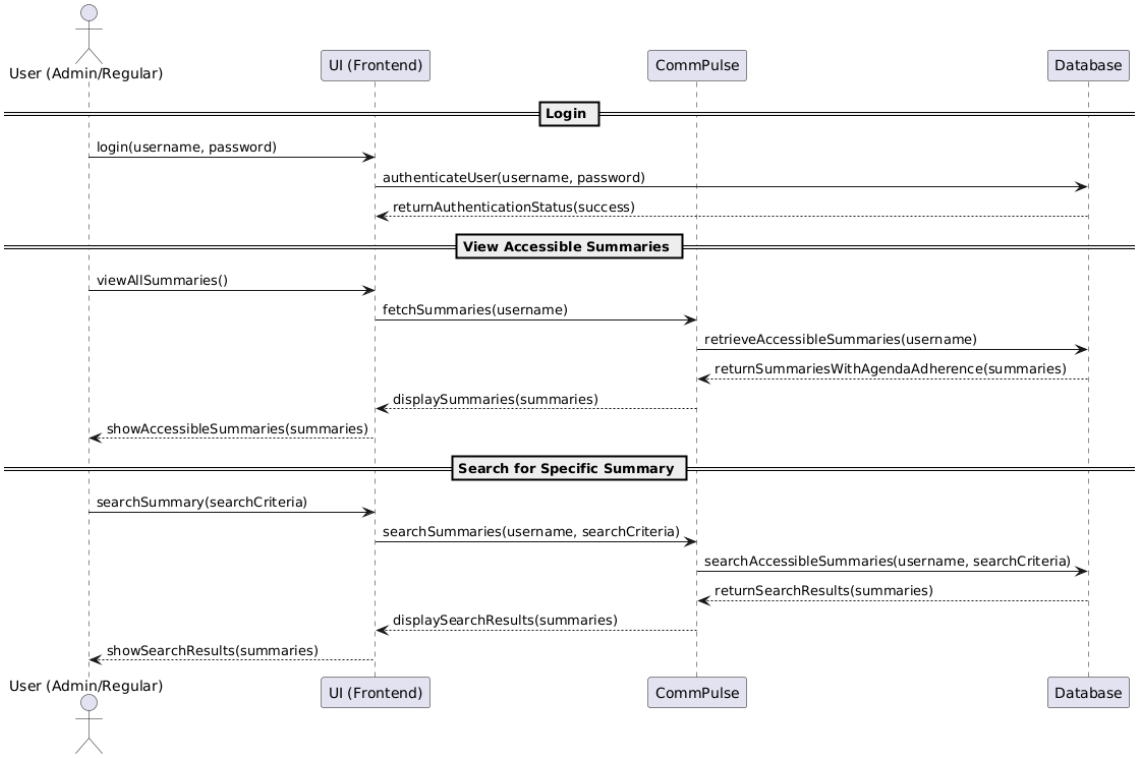

Figure 3.14: Viewing and searching meeting summaries - Sequence Diagram

3.2.5 State Transition Diagram

A state transition diagram for projects involving event handling and backend processes.

3.2.5.1 State Transition Diagram for Joining a Meeting

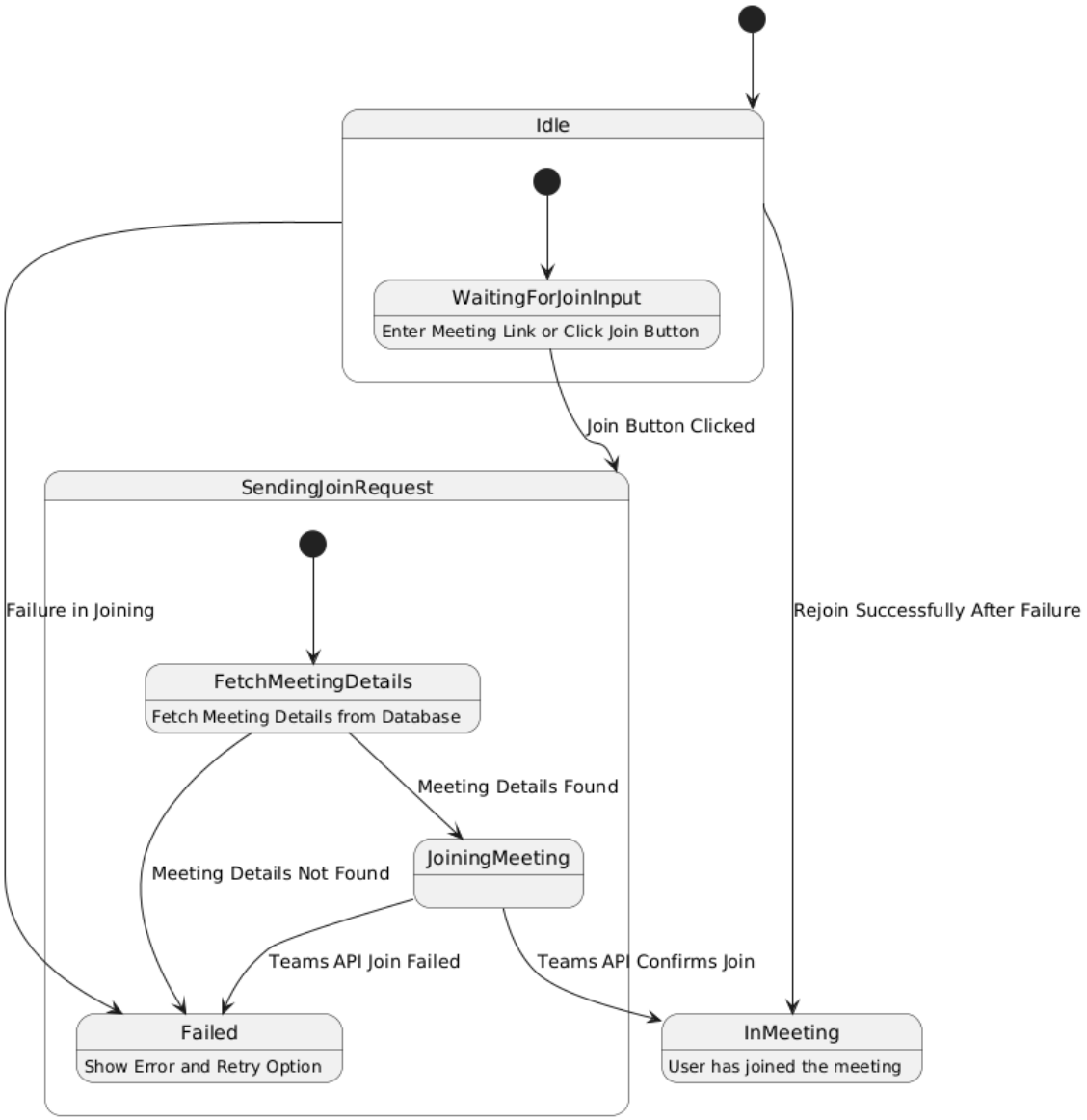

Figure 3.15: State Transition Diagram for Joining a Meeting

3.2.5.2 State Transition Diagram for Creating a Meeting

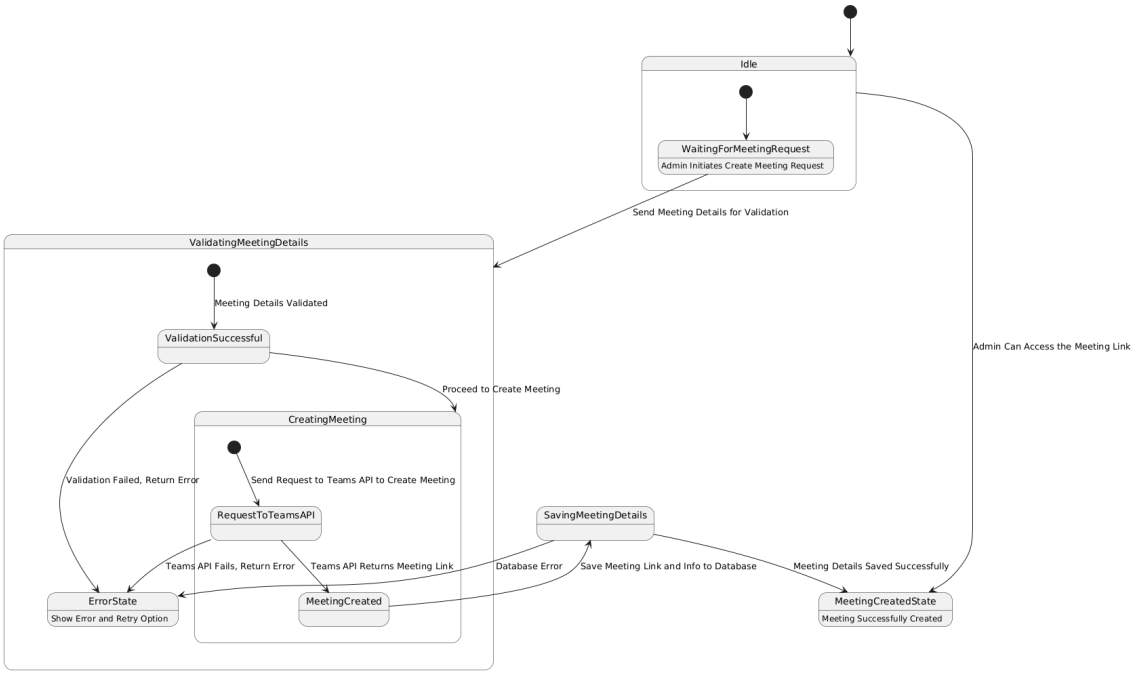

Figure 3.16: State Transition Diagram for Creating a Meeting

3.2.5.3 State Transition Diagram for Monitoring a Meeting

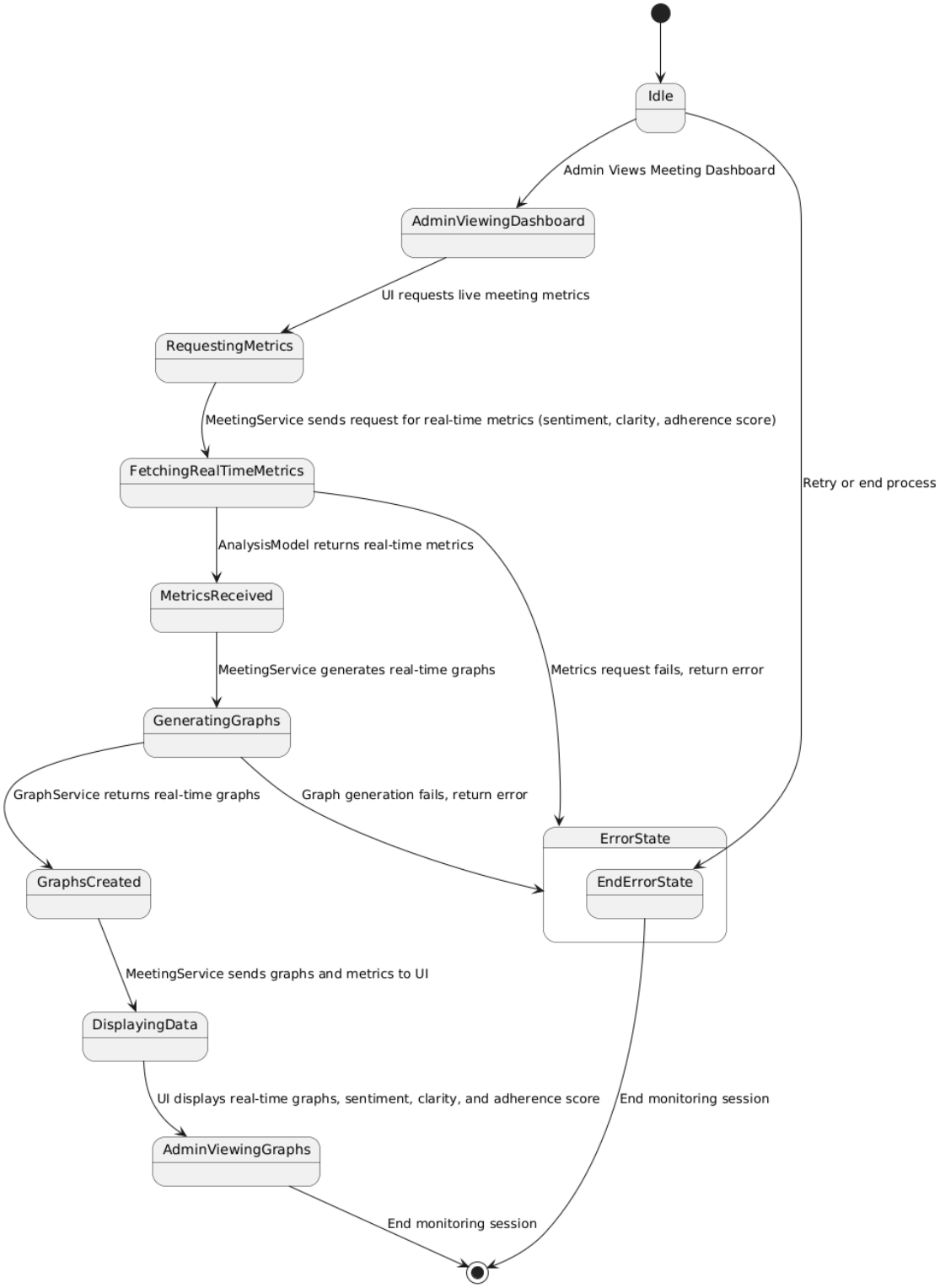

Figure 3.17: State Transition Diagram for Monitoring a Meeting

3.2.5.4 State Transition Diagram for Managing a Summary

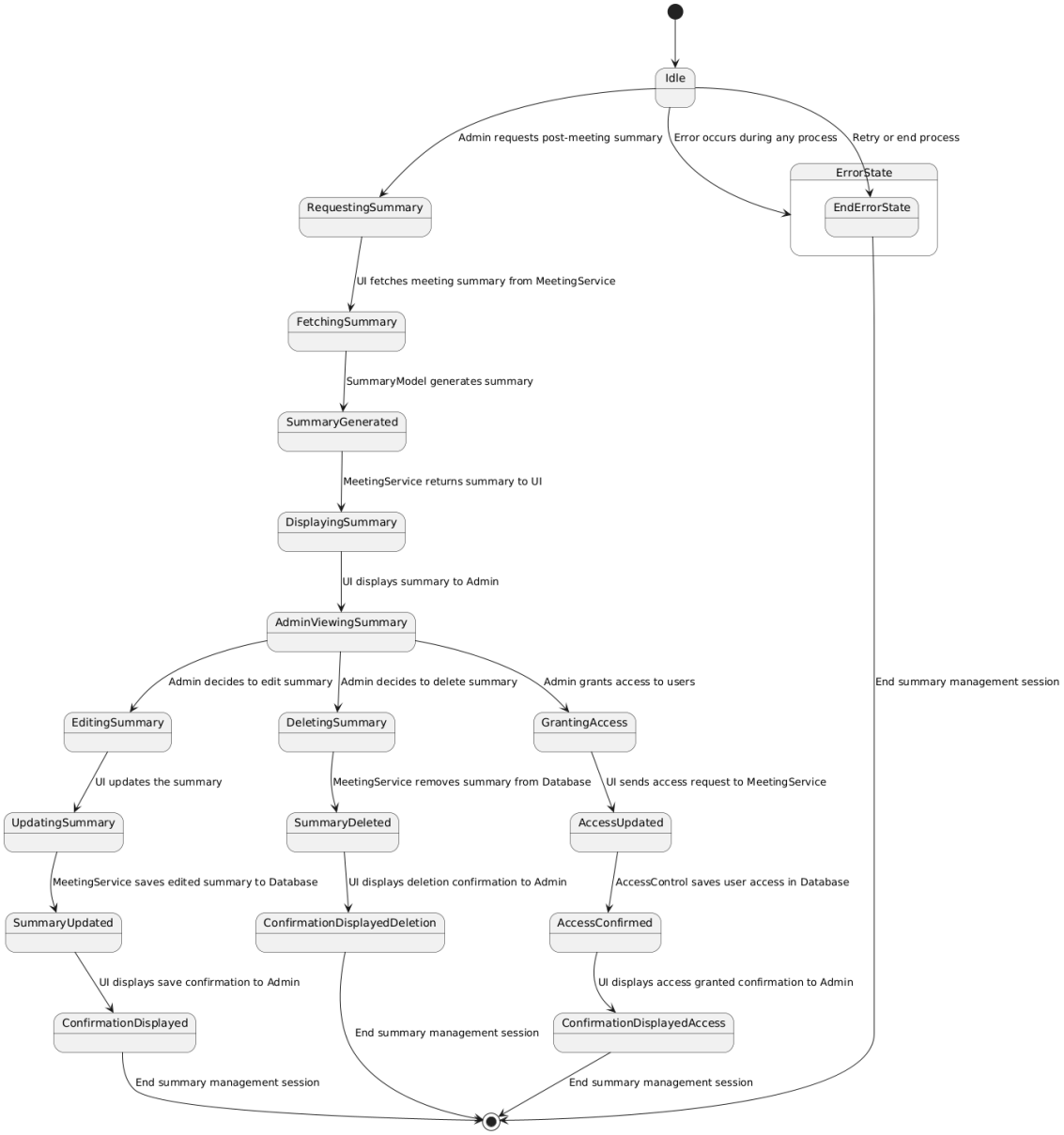

Figure 3.18: State Transition Diagram for Managing a Summary

3.2.5.5 State Transition Diagram for Viewing Post-Meeting Summary

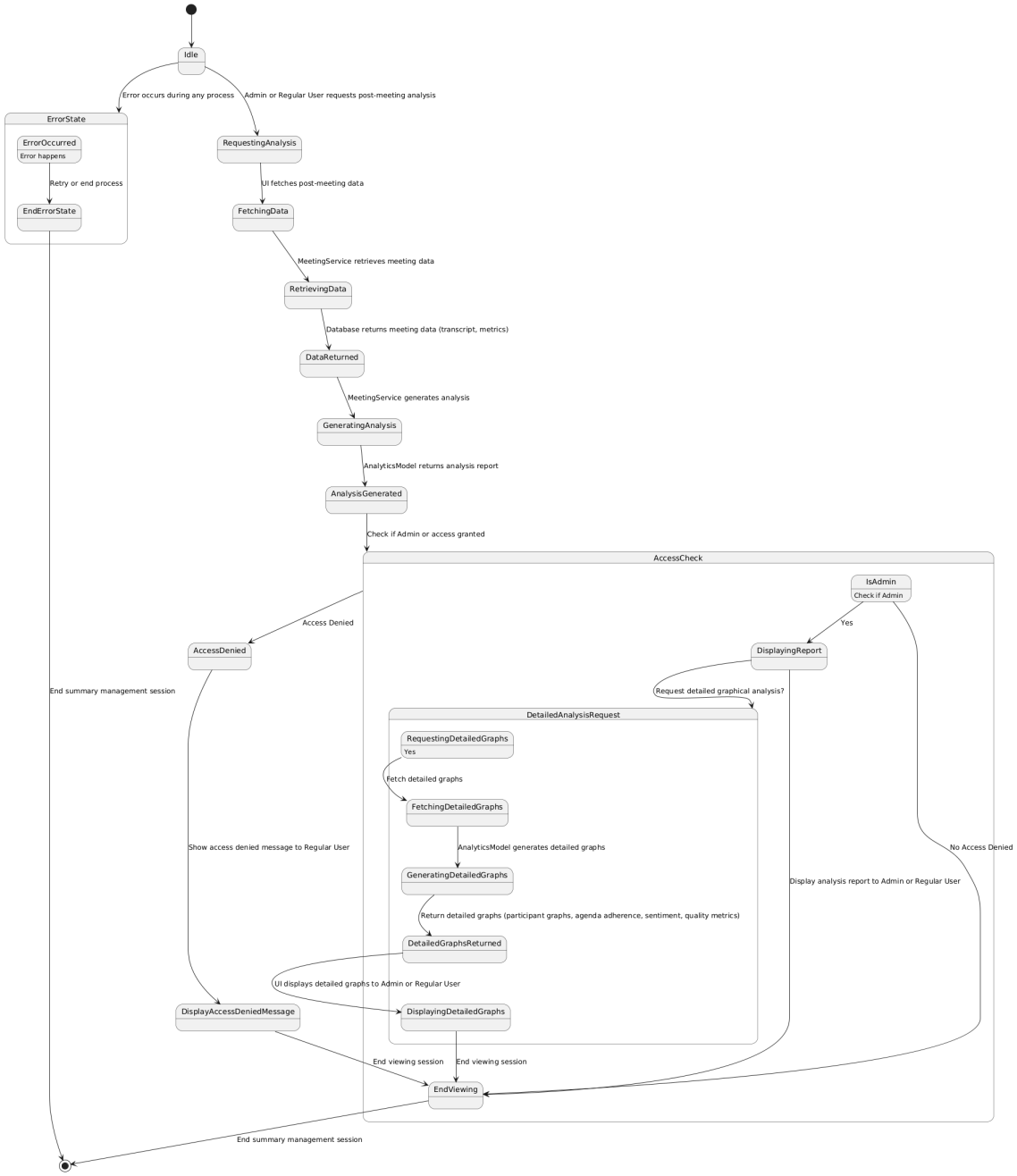

Figure 3.19: State Transition Diagram for Viewing Post-Meeting Summary

3.2.5.6 State Transition Diagram for Viewing and Searching Summary

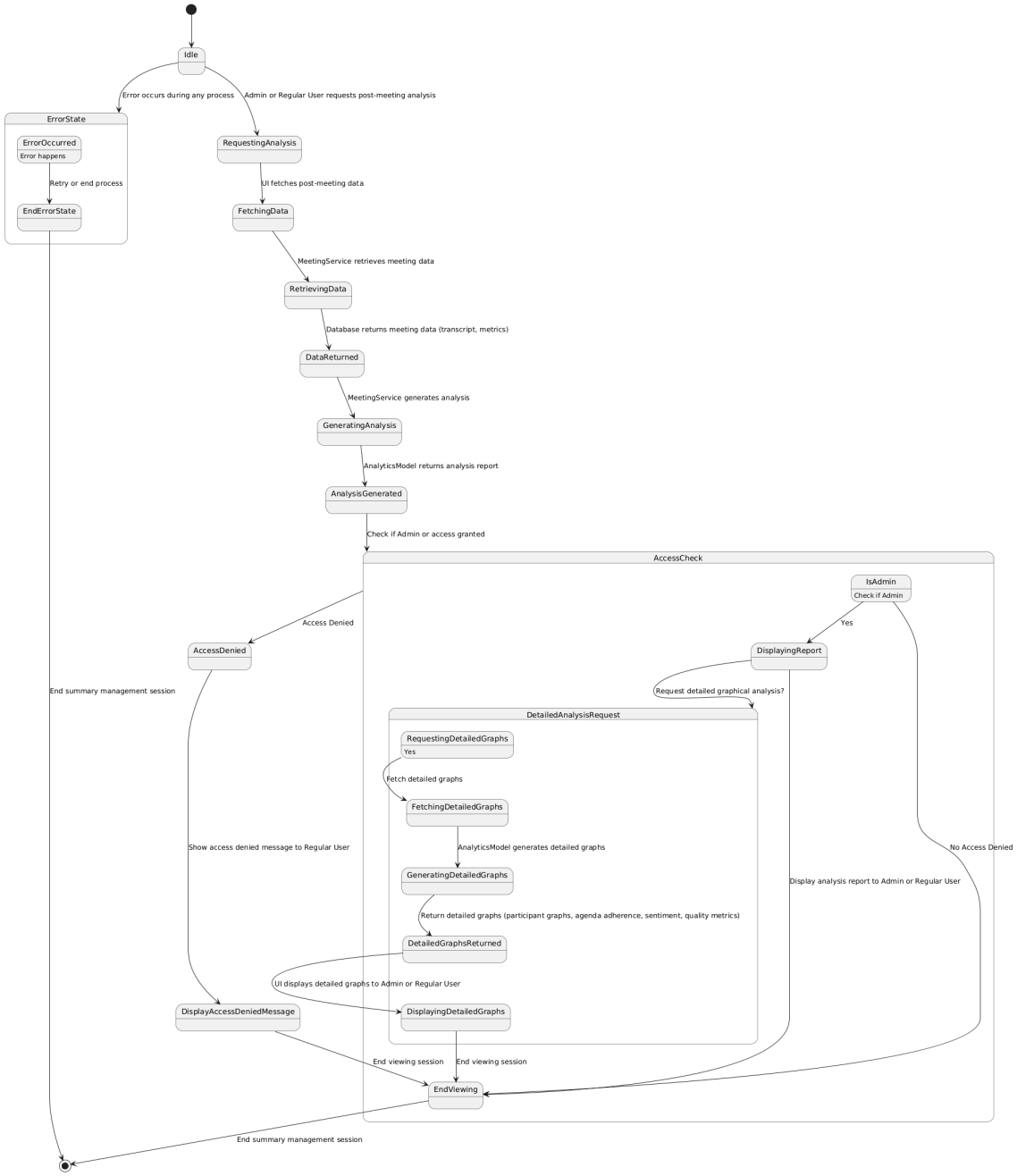

Figure 3.20: State Transition Diagram for Viewing and Searching Summary

3.3 Data Design

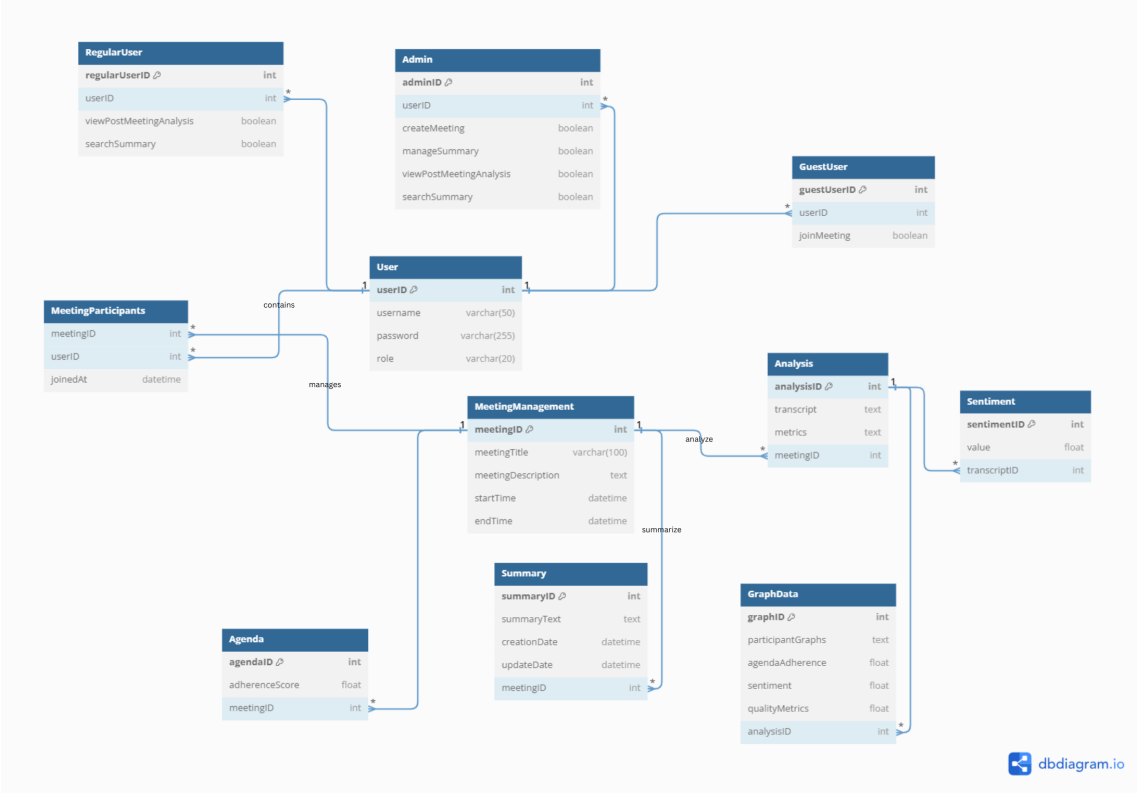

Figure 3.21: ERD Diagram of CommPulse System

3.4 Implementation

3.4.1 Figma Design

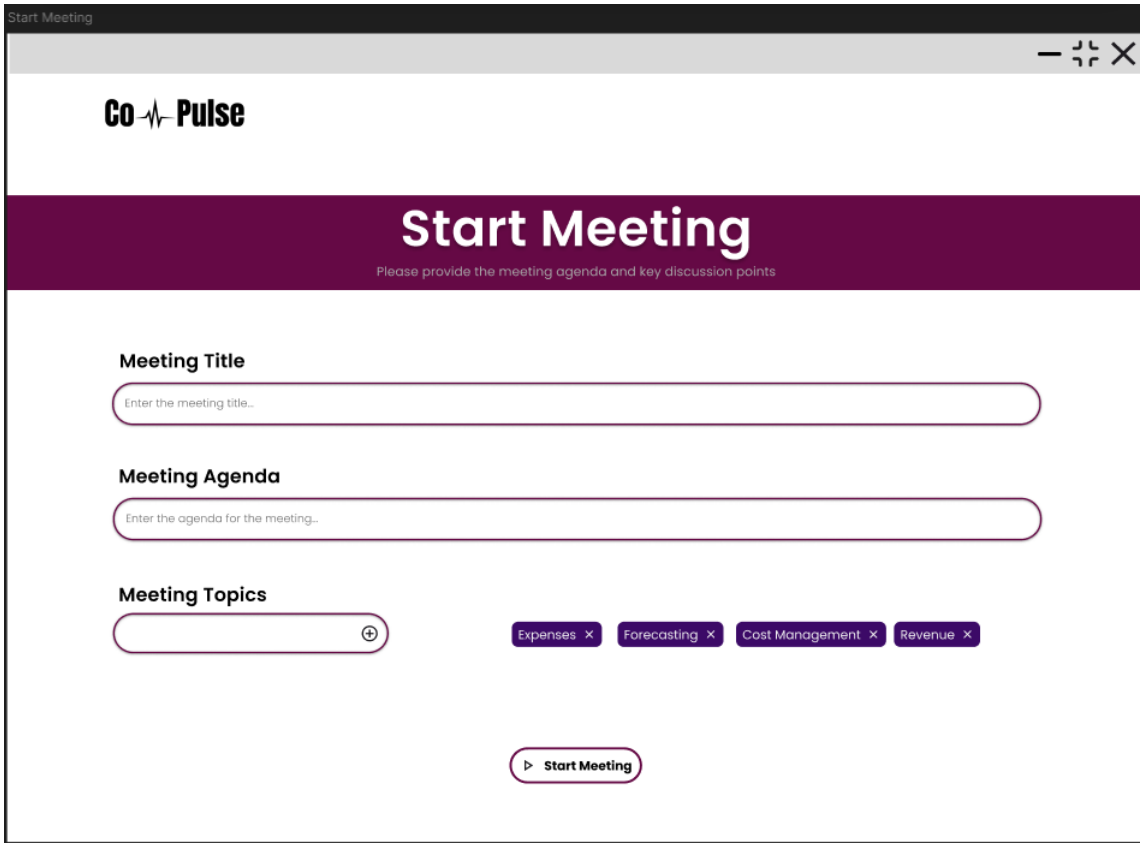

Figure 3.22: Starting Meeting Design

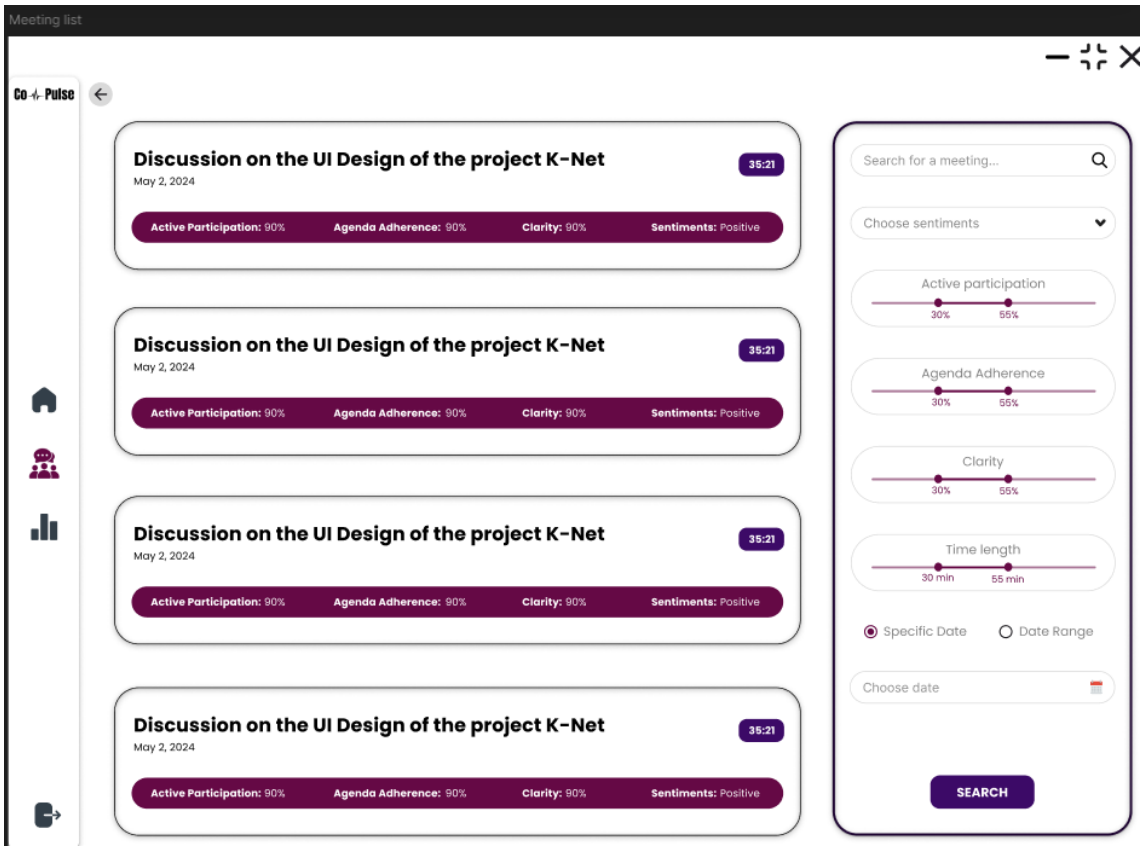

Figure 3.23: view and search summary

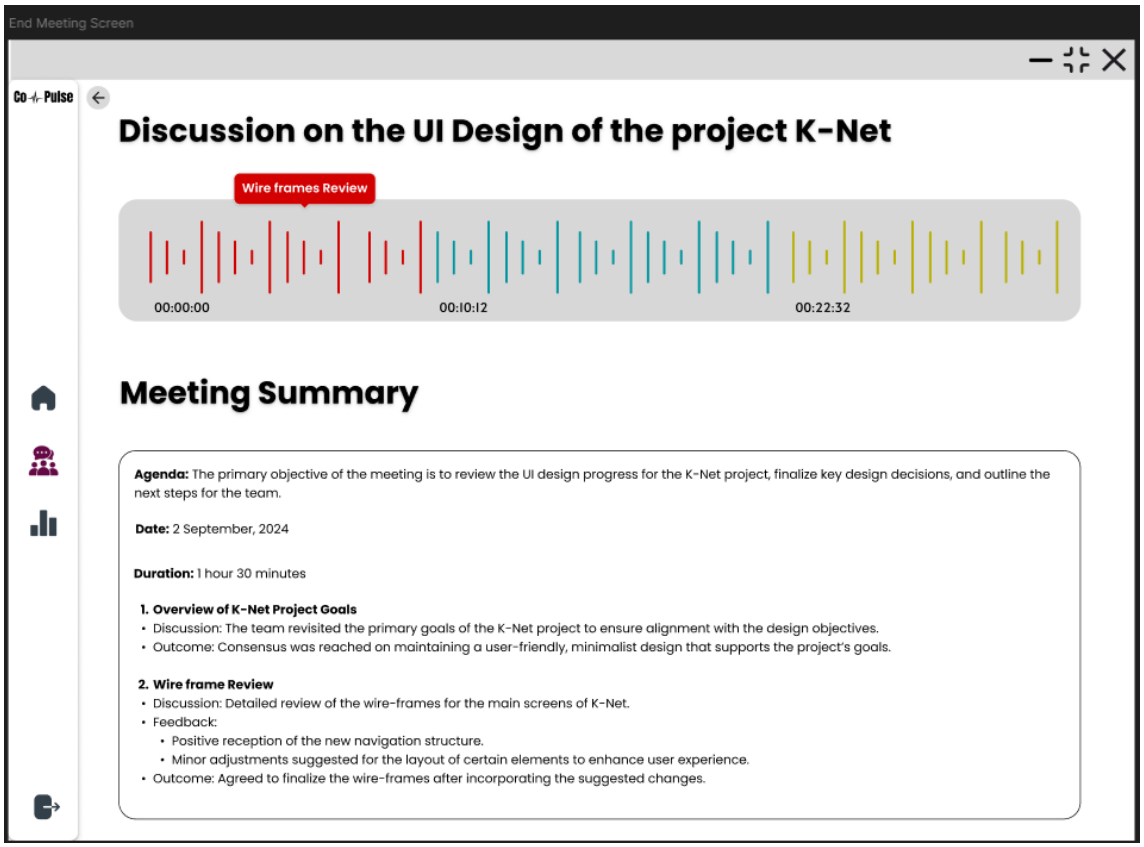

Figure 3.24: Getting Post meeting summary

3.4.2 Getting Live Transcript

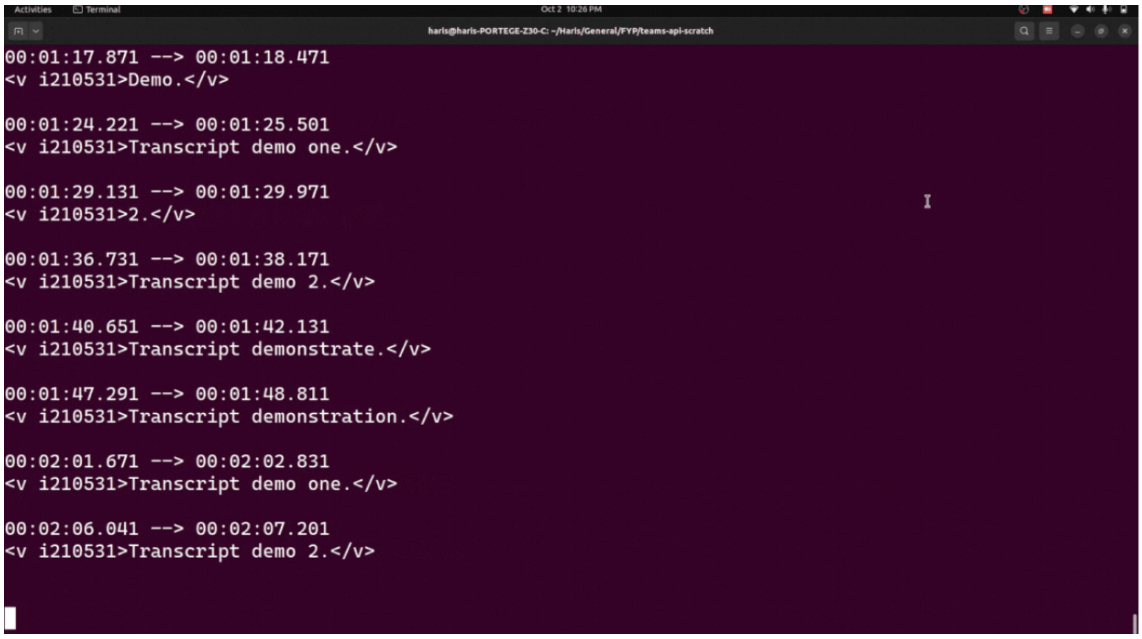

Figure 3.25: Fetching Live transcript from MS Teams API

Bibliography

[1] Alf Alferez. Increasing efficiency: How online meetings improve the workday. *ecommercenext*, 2023.

[2] Workhuman Editorial Team. How to make virtual meetings more interactive, fun, & engaging. *Workhuman*, 2024.

[3] B.G. Zulch. Communication: The foundation of project management. *Procedia Technology*, 2014.